



AGENTE RAG

Informe de la entrega

Arquitectura, decisiones, pruebas, benchmark, RAGAs y conclusiones

Nerea Aguilar Forés y Judit Espinoza Cervera

Índice

1. Introducción	2
2. Arquitectura elegida.....	2
3. Flujo de una consulta	2
4. Corpus y chunking.....	3
5. Decisiones tomadas	3
ChromaDB persistente	3
Prompt anti-alucinación	3
Query expansión.....	3
Benchmark con varios modelos	3
6. Cambios realizados durante las pruebas.....	4
7. Benchmark.....	4
8. Resultados observados	5
Aspectos positivos	5
Problemas detectados	5
9. RAGAs y métricas propias	5
Resultados RAGAs consolidados.....	5
Métricas propias.....	6
10. Análisis crítico, limitaciones y propuestas de mejora	6
11. Dificultades encontradas y como se resolvieron.....	7
12. Conclusiones	7

1. Introducción

El objetivo de la práctica ha sido construir un agente RAG capaz de responder preguntas sobre la asociación DNI Valencia a partir del corpus oficial proporcionado. El sistema combina recuperación de información y generación de lenguaje natural para ofrecer respuestas fundamentadas, con cita de fuentes y control anti-alucinación.

La solución entregada sigue el contrato de la práctica mediante la función `consultar(pregunta: str, conversation_id: str | None = None) -> dict`, situada en la raíz del repositorio. La salida incluye respuesta, fuentes, chunks recuperados, métricas y trazas.

2. Arquitectura elegida

Se ha elegido una arquitectura single-agent modular. No se implementa arquitectura hexagonal completa, porque el alcance final se ha centrado en consolidar las bandas 5, 6, 7 y 8 sin asumir el coste de una reestructuración completa a ports and adapters.

La lógica está separada en módulos:

- `chunker.py`: carga y división del corpus
- `embedder.py`: generación de embeddings
- `retriever.py`: consulta a ChromaDB
- `prompts.py`: prompt anti-alucinación
- `generator.py`: llamada al modelo generador
- `pipeline.py`: orquestación del flujo RAG

Esta estructura mantiene el código comprensible y facilita futuras ampliaciones, aunque no llega al desacoplamiento completo de ports y adapters requerido para Banda 10.

3. Flujo de una consulta

El flujo completo es el siguiente:

1. El usuario o corrector llama a `consultar.py`
2. `pipeline.answer` recibe la pregunta
3. `retriever.retrieve` transforma la pregunta en embedding y recupera chunks desde ChromaDB
4. `prompts.build_prompt` construye el prompt con el contexto recuperado y reglas de seguridad
5. `generator.generate` invoca el modelo configurado en Ollama o PoliGPT
6. El pipeline devuelve un diccionario con respuesta, fuentes, chunks y métricas

La respuesta final debe estar basada en los documentos recuperados. Si la información no aparece en el contexto, el prompt obliga al modelo a responder: No tengo esa información en mis fuentes.

4. Corpus y chunking

El corpus contiene 16 ficheros .txt sobre DNI Valencia. Incluye información general, desayunos solidarios, RESIS, COLES, horarios, contacto, preguntas frecuentes e impacto social.

Se usa RecursiveCharacterTextSplitter con:

- chunk_size=500
- chunk_overlap=100

Esta configuración permite conservar contexto suficiente dentro de cada fragmento y mantener prompts razonables. Cada chunk conserva el nombre del archivo origen, lo que permite citar fuentes en la salida.

5. Decisiones tomadas

ChromaDB persistente

Se usa ChromaDB persistente para evitar recalcular embeddings en cada ejecución. Esto mejora la velocidad en consultas repetidas y facilita la defensa oral.

Prompt anti-alucinación

El prompt se reforzó durante las pruebas para:

- responder solo con información del contexto
- rechazar preguntas fuera de ámbito
- citar siempre el archivo fuente
- no inventar datos numéricos
- no inventar significados de siglas
- separar conceptos cuando la pregunta sea comparativa

Query expansión

Se incorporó una expansión sencilla para preguntas que contienen RESIS o COLES:

- RESIS se asocia a residencias, mayores y abuelitos
- COLES se asocia a refuerzo escolar, niños y colegios

Esto mejora el retrieval en preguntas comparativas.

Benchmark con varios modelos

Para Banda 7 se compararon cuatro modelos manteniendo el mismo corpus, chunking, retrieval y prompt. Así se comparan modelos, no sistemas distintos.

6. Cambios realizados durante las pruebas

Las pruebas documentadas en pruebas/ muestran un proceso iterativo:

- Se corrigió un prompt inicial que todavía arrastraba el dominio de la plantilla
- Se reforzó el comportamiento anti-alucinación
- Se aumento el número de chunks recuperados para mejorar preguntas multi-documento
- Se añadió expansión de consulta para RESIS y COLES
- Se ajustó el prompt para evitar generalizaciones
- Se añadieron reglas para no inventar significados de siglas
- Se mejoró el formato de comparativas
- Se ejecutaron benchmarks con modelos locales y PoliGPT

Estos cambios mejoraron especialmente las respuestas sobre RESIS y COLES, que al principio tendían a mezclar conceptos o inventar significados.

7. Benchmark

El benchmark se realizó con 8 preguntas:

- preguntas generales sobre DNI
- preguntas sobre desayunos solidarios
- comparativas entre RESIS y COLES
- preguntas sobre refuerzo escolar
- preguntas sobre residencias
- preguntas fuera de ámbito

Modelos evaluados:

Modelo	Entorno	Resultado cualitativo
llama3.2:3b	Ollama local	Rápido y funcional, pero limitado en síntesis compleja.
qwen2.5:3b	Ollama local	Mejor equilibrio local entre calidad, velocidad y español.
poligpt	PoliGPT UPV	Respuestas más completas y mejor estructuradas, con mayor latencia.
qwen	PoliGPT UPV	Buena síntesis, pero respuestas largas y latencia elevada.

Los resultados completos están en [benchmark/benchmark.md](#), [benchmark/benchmark.json](#) y los JSON de [benchmark/runs/](#).

8. Resultados observados

Aspectos positivos

- El agente responde correctamente preguntas específicas cuando el retrieval encuentra contexto suficiente
- Las preguntas fuera de ámbito se rechazan con la frase esperada
- Las comparativas entre RESIS y COLES mejoraron tras introducir reglas de prompt y expansión de consulta
- Los modelos PoliGPT produjeron respuestas más estructuradas
- qwen2.5:3b fue una alternativa local equilibrada

Problemas detectados

- La pregunta "Que es DNI?" fue rechazada o respondida de forma insuficiente en algunos modelos, aunque había contexto parcial
- Los modelos pequeños pueden mezclar información cuando la pregunta requiere síntesis multi-documento
- Algunos modelos externos generan respuestas demasiado largas, aumentando latencia y tokens
- Los resultados muestran que recuperar las fuentes esperadas no siempre garantiza una respuesta perfecta: algunos modelos recuperan buenos chunks pero aun así responden de forma conservadora o mezclan conceptos

9. RAGAs y métricas propias

Para Banda 8 se ha incorporado una evaluación adicional en evaluacion/.

Los artefactos principales son:

- evaluacion/ragas_results.json: resultados de faithfulness, answer_relevancy, context_precision y context_recall
- evaluacion/metricas_propias.md: definición y valores de dos métricas propias
- evaluacion/ground_truth.json: respuestas de referencia usadas para comparar las preguntas
- evaluacion/run_ragas_eval.py: script preparado para recalcular métricas si se incorporan los JSON crudos de benchmark/runs/

Resultados RAGAs consolidados

Modelo	Faithfulness	Answer relevancy	Context precision	Context recall
llama3.2:3b	0.691	0.661	0.602	1.00
qwen2.5:3b	0.706	0.653	0.602	1.00
poligpt	0.691	0.588	0.602	1.00
qwen	0.716	0.637	0.602	1.00

Promedios globales:

- faithfulness: 0.701
- answer_relevancy: 0.635
- context_precision: 0.602
- context_recall: 1.00

La evaluación muestra que el sistema es bastante fiel al contexto recuperado y que el principal margen de mejora está en aumentar la precisión/recall del contexto para preguntas generales o comparativas.

Métricas propias

Se definieron dos métricas propias:

1. Tasa de rechazo correcto fuera de ámbito
2. Cobertura de fuentes esperadas

Resultados:

Modelo	Rechazo correcto	Cobertura de fuentes
llama3.2:3b	1.00	1.00
qwen2.5:3b	1.00	1.00
poligpt	1.00	1.00
qwen	1.00	1.00

La tasa de rechazo correcto confirma que el prompt anti-alucinación funciona bien en las preguntas fuera de ámbito. La cobertura de fuentes muestra que PoliGPT y qwen recuperan o usan mejor las fuentes esperadas, aunque qwen2.5:3b sigue siendo la alternativa local más equilibrada.

Nota metodológica: los cuatro modelos tienen runs reales en benchmark/runs/. Los runs locales de llama3.2:3b y qwen2.5:3b se han generado de nuevo con Ollama el 2026-05-30. Los runs de PoliGPT se han incorporado desde los resultados guardados por el equipo. El script evaluacion/run_ragas_eval.py permite recalcular una versión heurística reproducible.

10. Análisis crítico, limitaciones y propuestas de mejora

- La arquitectura no es hexagonal
- No hay retrieval híbrido con BM25
- No hay re-ranking
- No hay memoria conversacional real
- El sistema depende de que el índice ChromaDB se haya construido previamente
- La calidad final depende mucho del modelo elegido
- El control anti-alucinación puede provocar rechazos conservadores

Como propuestas de mejora, se plantea incorporar retrieval híbrido con BM25 para reforzar búsquedas por términos exactos, añadir re-ranking para ordenar mejor los chunks recuperados, agrupar pares Q:/A: antes del chunking para no cortar preguntas y respuestas, y refactorizar a arquitectura hexagonal si se quisiera aspirar a Banda 10. También sería recomendable revisar el prompt de comparativas para reducir errores como la confusión entre RESIS y COLES en algunos modelos.

11. Dificultades encontradas y como se resolvieron

- Adaptar la plantilla inicial al dominio real de DNI Valencia. Se resolvió reescribiendo el rol del asistente, el README y la documentación para eliminar referencias al caso anterior
- Evitar que el modelo inventara significados para siglas como RESIS. Se resolvió añadiendo reglas explícitas al prompt para no inventar acrónimos y una regla específica para RESIS y COLES
- Conseguir buenas respuestas comparativas con modelos pequeños. Se resolvió aumentando el top-k, añadiendo query expansión y forzando respuestas separadas por conceptos
- Mantener coherencia entre benchmark, features.json y documentación. Se resolvió centralizando las bandas declaradas en features.json y añadiendo los runs reales en benchmark/runs/
- Comparar modelos con latencias y estilos de respuesta muy distintos. Se resolvió usando el mismo set de preguntas, el mismo pipeline y métricas comunes para todos los modelos

12. Conclusiones

El sistema cumple el objetivo principal de la práctica: responder preguntas sobre DNI Valencia usando un pipeline RAG con fuentes y control anti-alucinacion. La solución es especialmente sólida para Banda 8, porque incluye métricas por respuesta, benchmark con cuatro modelos, evaluacion RAGAs y dos métricas propias.

La mejor opción local observada es qwen2.5:3b, por su equilibrio entre calidad y velocidad. PoliGPT ofrece respuestas más completas, pero con mayor latencia. La evaluacion de Banda 8 confirma que el sistema es fiel al contexto y rechaza bien preguntas fuera de ámbito.